PLAN DE PRUEBAS FUNCIONALES - MÓDULO DE AUTENTICACIÓN

1. Introducción **2**
2. Alcance de la Prueba **2**
3. Ambientes de Prueba **2**
4. Criterios de Aceptación **3**
5. Estrategia de Pruebas **3**
5.1 Tipos de Pruebas a Realizar 3
1. Pruebas de Solicitud Correcta (Happy Path): 3
2. Pruebas de Error (Validación de Errores): 3
3. Pruebas de Autenticación y Autorización: 3
4. Criterios de Aceptación **3**
1. Códigos de Estado HTTP 4
2. Estructura de la Respuesta (JSON) 4
3. Headers 4
4. Seguridad 4
5. Tiempo de Respuesta 4
5. Estrategia de Pruebas **4**
5.1 Tipos de Pruebas a Realizar 5
5.2 Herramientas de Ejecución 6
6. Detalles de la Prueba por Módulo **6**
6.1 Módulo de Autenticación 6
6.1.1 Casos de Flujo Exitoso y Validación de Errores 6
6.1.2 Casos de Autenticación y Seguridad 10
6.2 Módulo de Usuarios 11
6.2.1 Casos de Flujo Exitoso 11
6.2.2 Casos de Validación de Errores 14
6.2.3 Casos de Autenticación y Seguridad 16
6.3 Módulo de Gastos 16
6.3.1 Casos de Flujo Exitoso 16
6.3.2 Casos de Validación de Errores 19
6.4 Módulo de Ingresos 22
6.4.1 Casos de Flujo Exitoso 22
6.4.2 Casos de Validación de Errores 25
6.4.3 Casos de Autenticación y Seguridad 26
6.5 Módulo de Bancos 27
6.5.1 Casos de Flujo Exitoso 27
6.5.2 Casos de Validación de Errores 30
6.5.3 Casos de Autenticación y Seguridad 31
6.6 Módulo de Categorías 32
6.6.1 Casos de Flujo Exitoso 32
6.6.2 Casos de Validación de Errores 35

6.6.3 Casos de Autenticación y Seguridad..... 36
7. Reporte de Resultados..... 37
7.1 Módulo de Autenticación..... 37
7.2 Módulo de Usuarios..... 38
7.3 Módulo de Gastos..... 39
7.4 Módulo de Ingresos..... 40
7.5 Módulo de Bancos..... 41
7.6 Módulo de Categorías..... 42
8. Conclusión..... 42

1. Introducción

- **Nombre del Proyecto:** Sistema de Gestión Financiera Personal.
- **Funcionalidad a probar:** Módulo de autenticación y acceso (API REST).
- **Descripción general:** Este módulo gestiona las operaciones de inicio de sesión, cierre de sesión y registro/acceso de usuarios. Su objetivo es controlar la seguridad del sistema y garantizar que solo los usuarios autenticados accedan a la información financiera.

2. Alcance de la Prueba

- **Objetivo:** Validar que los endpoints del módulo de autenticación procesen correctamente las solicitudes exitosas y gestionen adecuadamente los errores y la seguridad.
- **Endpoints a probar:**

    - POST /api/v1/auth/register — Registro de nuevos usuarios.
    - POST /api/v1/auth/login — Inicio de sesión y generación de token JWT.
    - POST /api/v1/auth/logout — Cierre de sesión e invalidación de token.

3. Ambientes de Prueba

- **Entorno de Desarrollo:** https://dev.api.misitio.com

- **Entorno de Staging (Pruebas):** https://staging.api.misitio.com

- **Entorno de Producción:** https://api.misitio.com

4. Criterios de Aceptación

Para considerar una prueba como **exitosa**, se deben cumplir las siguientes condiciones:

- **Códigos de estado HTTP:** Retorno adecuado del código según el escenario (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized).
- **Estructura JSON:** La respuesta debe coincidir exactamente con el esquema especificado en el documento OpenAPI.
- **Headers:** Envío y recepción correcta del header Authorization: Bearer y Content-Type: application/json.
- **Seguridad:** Los errores de autenticación deben retornar mensajes genéricos por seguridad (ej. *"Credenciales inválidas"*).

5. Estrategia de Pruebas

5.1 Tipos de Pruebas a Realizar

1. **Pruebas de Solicitud Correcta (Happy Path):**
    - Verificar el registro de usuario con datos válidos.
    - Validar inicio de sesión correcto con retorno de token JWT activo.
    - Confirmar cierre de sesión exitoso con token válido.
2. **Pruebas de Error (Validación de Errores):**
    - Envío de cuerpo de solicitud incompleto o con parámetros faltantes.
    - Formato de correo electrónico inválido.
    - Intento registrarme con un correo ya existente en el sistema.
3. **Pruebas de Autenticación y Autorización:**
    - Intento de login con contraseña o correo incorrecto.
    - Petición de logout sin incluir el token Bearer en el header.
    - Intento de reuso de un token de sesión que ya fue finalizado.

4. Criterios de Aceptación

Para considerar una prueba como exitosa dentro del Módulo de Autenticación, se deben cumplir las siguientes condiciones mínimas:

1. Códigos de Estado HTTP

La API debe retornar el código de estado HTTP correspondiente según el escenario probado:

- 200 OK: Para solicitudes exitosas de inicio de sesión y cierre de sesión.
- 201 Created: Para el registro exitoso de un nuevo usuario.
- 400 Bad Request: Para solicitudes con datos inválidos, campos faltantes o correos ya registrados.
- 401 Unauthorized: Para intentos de acceso con credenciales incorrectas o sin token de autorización.

2. Estructura de la Respuesta (JSON)

El cuerpo de la respuesta debe cumplir con la estructura definida en la especificación OpenAPI del módulo. Esto implica:

- Que los nombres de los campos coincidan exactamente (por ejemplo: token, email, nombre, error).
- Que los tipos de datos sean los correctos (String, Integer, boolean, etc.).
- Que no se incluyan campos adicionales no contemplados.
- Que no se exponga información sensible, como la contraseña del usuario, en ninguna respuesta.

3. Headers

Las solicitudes y respuestas deben manejar correctamente los siguientes encabezados:

- Content-Type: application/json en todas las solicitudes que envíen cuerpo.
- Authorization: Bearer <token> en los endpoints protegidos (como el cierre de sesión).
- La respuesta debe incluir el header Content-Type: application/json.

4. Seguridad

Los mensajes de error relacionados con la autenticación deben ser genéricos, evitando revelar información sensible al cliente. Por ejemplo:

- Correcto: "Credenciales inválidas"
- Incorrecto: "La contraseña es incorrecta" o "El usuario no existe"

Esto aplica especialmente a los casos de login fallido, donde el mensaje no debe permitir inferir si el correo existe o no en el sistema.

5. Tiempo de Respuesta

La API debe responder en un tiempo razonable (menor a 2 segundos) bajo condiciones normales de operación, garantizando una experiencia de usuario aceptable.

5. Estrategia de Pruebas

La estrategia de pruebas para el Módulo de Autenticación se basa en la validación funcional de los endpoints expuestos por la API, verificando tanto los flujos exitosos como los escenarios de error y las restricciones de seguridad. Las pruebas se ejecutarán de forma manual utilizando Postman como herramienta principal, siguiendo el orden definido en los casos de prueba y verificando en cada uno los criterios de aceptación establecidos.

El objetivo de esta estrategia es garantizar que el módulo de autenticación responda correctamente ante las solicitudes de los usuarios, gestione adecuadamente los errores y proteja la información sensible del sistema.

5.1 Tipos de Pruebas a Realizar

Con el fin de cubrir de manera integral el comportamiento del módulo, se ejecutarán los siguientes tipos de pruebas:

- ➔ 1. Pruebas de Solicitud Correcta (Happy Path)

Este tipo de pruebas verifica que la API responda correctamente cuando recibe solicitudes válidas, es decir, cuando el usuario envía todos los datos requeridos con el formato adecuado. Se busca confirmar que las operaciones principales del módulo funcionen según lo esperado.

Casos incluidos:

- ❖ Verificar el registro de usuario con datos válidos.
- ❖ Validar inicio de sesión correcto con retorno de token JWT activo.
- ❖ Confirmar cierre de sesión exitoso con token válido.

- ➔ 2. Pruebas de Error (Validación de Errores)

Este tipo de pruebas evalúa la capacidad de la API para manejar solicitudes incorrectas o incompletas, retornando los códigos de estado y mensajes de error correspondientes. Se verifica que el sistema no procesa datos inválidos y que informe adecuadamente al cliente sobre el problema.

Casos incluidos:

- ❖ Envío de cuerpo de solicitud incompleto o con parámetros faltantes.
- ❖ Formato de correo electrónico inválido.
- ❖ Intentar registrar con un correo ya existente en el sistema.

➔ 3. Pruebas de Autenticación y Autorización

Este tipo de pruebas se enfoca en validar los mecanismos de seguridad del módulo, asegurando que solo los usuarios autenticados puedan acceder a los recursos protegidos y que los tokens de sesión sean gestionados correctamente.

Casos incluidos:

- ❖ Intento de login con contraseña o correo incorrecto.
- ❖ Petición de logout sin incluir el token Bearer en el header.
- ❖ Intento de reuso de un token de sesión que ya fue finalizado.

5.2 Herramientas de Ejecución

- ❖ Postman: herramienta principal para el envío de solicitudes HTTP y la verificación de respuestas.
- ❖ JWT.io: utilizada para validar la estructura y vigencia de los tokens JWT generados.
- ❖ Navegador web / editor de texto: para el análisis manual de las respuestas JSON.

6. Detalles de la Prueba por Módulo

A continuación se documentan los casos de prueba diseñados para validar el comportamiento de cada módulo del Sistema de Gestión Financiera Personal. Cada caso especifica el endpoint a consumir, el método HTTP, los headers requeridos, el cuerpo de la solicitud, la respuesta esperada y la condición que determina si la prueba se considera exitosa.

La sección está organizada por módulos funcionales del sistema. Cada módulo contiene sus propios casos de prueba, agrupados según el tipo de validación: flujo exitoso (Happy Path), validación de errores y seguridad.

6.1 Módulo de Autenticación

Este módulo cubre las operaciones de registro, inicio de sesión y cierre de sesión de usuarios. Los casos se dividen en dos grupos: flujos exitosos y validaciones de seguridad.

6.1.1 Casos de Flujo Exitoso y Validación de Errores

Caso de Prueba 1: Registro exitoso de un nuevo usuario

- Endpoint: POST /api/v1/auth/register
- Método HTTP: POST
- Descripción: Registrar un nuevo usuario con datos válidos en el sistema.
- Parámetros: Ninguno.
- Headers:
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{
"nombre": "Carlos Pérez",
"email": "carlos.perez@email.com",
"password": "SecureP@ss123"
}

- Código de Respuesta Esperado: 201 Created
- Cuerpo de la Respuesta Esperada:

{
"id": 1,
"nombre": "Carlos Pérez",
"email": "carlos.perez@email.com",
"estado": "ACTIVO"
}

- Prueba exitosa si: La respuesta devuelve el código 201 Created, los datos del usuario registrado coinciden con los enviados y no se expone la contraseña.

Caso de Prueba 2: Inicio de sesión exitoso

- Endpoint: POST /api/v1/auth/login
- Método HTTP: POST
- Descripción: Iniciar sesión con credenciales válidas y obtener el token JWT.
- Parámetros: Ninguno.
- Headers:
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"email": "carlos.perez@email.com",

"password": "SecureP@ss123"

}

- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",

"tokenType": "Bearer",

"expiresIn": 3600,

"user": {

"id": 1,

"nombre": "Carlos Pérez",

"email": "carlos.perez@email.com"

}

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK, el token JWT es válido y no está expirado, y el objeto user corresponde a las credenciales enviadas.

Caso de Prueba 3: Cierre de sesión exitoso

- Endpoint: POST /api/v1/auth/logout
- Método HTTP: POST
- Descripción: Cerrar sesión con un token JWT válido e invalidar.
- Parámetros: Ninguno.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"message": "Sesión cerrada exitosamente"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK y el token utilizado queda invalidado, de modo que no puede reutilizarse.

Caso de Prueba 4: Error al registrar usuario con correo ya existente

- Endpoint: POST /api/v1/auth/register
- Método HTTP: POST
- Descripción: Intentar registrar un usuario con un correo electrónico que ya está registrado en el sistema.
- Parámetros: Ninguno.
- Headers:
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"nombre": "Otro Usuario",

"email": "carlos.perez@email.com",

"password": "AnotherP@ss456"

}

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

{

"error": "El correo electrónico ya está registrado"

}

- Prueba exitosa si: La API devuelve un código 400 Bad Request con un mensaje de error indicando que el correo ya existe.

Caso de Prueba 5: Error al registrar usuario con correo electrónico inválido

- Endpoint: POST /api/v1/auth/register
- Método HTTP: POST
- Descripción: Intentar registrar un usuario con un formato de correo electrónico inválido.
- Parámetros: Ninguno.
- Headers:
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"nombre": "Usuario Inválido",

"email": "correo-invalido",

"password": "SecureP@ss123"

}

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

{

"error": "El formato del correo electrónico no es válido"

}

- Prueba exitosa si: La API devuelve un código 400 Bad Request con un mensaje de error indicando que el formato del correo no es válido.

Caso de Prueba 6: Error al registrar usuario con campos faltantes

- Endpoint: POST /api/v1/auth/register
- Método HTTP: POST
- Descripción: Intentar registrar un usuario sin enviar el campo password.
- Parámetros: Ninguno.

- Headers:
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"nombre": "Usuario Sin Password",

"email": "sinpassword@email.com"

}

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

{

"error": "El campo password es obligatorio"

}

- Prueba exitosa si: La API devuelve un código 400 Bad Request con un mensaje de error indicando que el campo password es obligatorio.

6.1.2 Casos de Autenticación y Seguridad

Caso de Prueba 7: Login con credenciales incorrectas

- Endpoint: POST /api/v1/auth/login
- Método HTTP: POST
- Descripción: Verificar que no se permite iniciar sesión con credenciales inválidas.
- Headers: Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{
"email": "carlos.perez@email.com",
"password": "WrongPassword123"
}

- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{
"error": "Credenciales inválidas"
}

- Prueba exitosa si: La API devuelve 401 Unauthorized con un mensaje genérico, sin revelar si el correo o la contraseña son incorrectos.

Caso de Prueba 8: Logout sin token Bearer

- Endpoint: POST /api/v1/auth/logout
- Método HTTP: POST
- Descripción: Verificar que no se puede cerrar sesión sin incluir el token de autorización.
- Headers: Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{
"error": "Token de autorización no proporcionado"
}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que se requiere autenticación.

Caso de Prueba 9: Reuso de un token finalizado

- Endpoint: POST /api/v1/auth/logout
- Método HTTP: POST
- Descripción: Intentar usar un token JWT que ya fue invalidado tras un cierre de sesión previo.
- Headers: Authorization: Bearer <token_invalidado>
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{
"error": "Token inválido o expirado"
}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que el token ya no es válido.

6.2 Módulo de Usuarios

Este módulo cubre las operaciones de consulta, actualización y eliminación de usuarios registrados en el sistema. Los casos se dividen en tres grupos: flujos exitosos, validación de errores y seguridad.

6.2.1 Casos de Flujo Exitoso

Caso de Prueba 10: Listar todos los usuarios

- Endpoint: GET /api/v1/usuarios
- Método HTTP: GET
- Descripción: Obtener la lista completa de usuarios registrados en el sistema.
- Parámetros: Ninguno.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

[

{

"id": 1,

"nombre": "Carlos Pérez",

"email": "carlos.perez@email.com",

"estado": "ACTIVO",

"rol": "USER"

},

{

"id": 2,

"nombre": "Ana Gómez",

"email": "ana.gomez@email.com",

"estado": "ACTIVO",

"rol": "ADMIN"

}

]

- Prueba exitosa si: La respuesta devuelve el código 200 OK y una lista con todos los usuarios registrados, sin exponer información sensible.

Caso de Prueba 11: Obtener usuario por ID existente

- Endpoint: GET /api/v1/usuarios/{id}
- Método HTTP: GET
- Descripción: Obtener la información de un usuario específico mediante su ID.
- Parámetros:
  - Path: id (integer) → ID del usuario a consultar.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"id": 1,

"nombre": "Carlos Pérez",

"email": "carlos.perez@email.com",

"estado": "ACTIVO",

"rol": "USER"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK con los datos del usuario correspondiente al ID enviado.

Caso de Prueba 12: Actualizar datos de un usuario

- Endpoint: PUT /api/v1/usuarios/{id}
- Método HTTP: PUT
- Descripción: Actualizar la información de un usuario existente.
- Parámetros:
  - Path: id (integer) → ID del usuario a actualizar.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"nombre": "Carlos Pérez Actualizado",

"email": "carlos.nuevo@email.com"

}

- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"id": 1,

"nombre": "Carlos Pérez Actualizado",

"email": "carlos.nuevo@email.com",

"estado": "ACTIVO",

"rol": "USER"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK con los datos actualizados del usuario.

Caso de Prueba 13: Eliminar un usuario

- Endpoint: DELETE /api/v1/usuarios/{id}
- Método HTTP: DELETE
- Descripción: Eliminar un usuario del sistema mediante su ID.
- Parámetros:
  - Path: id (integer) → ID del usuario a eliminar.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"message": "Usuario eliminado exitosamente"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK y el usuario ya no aparece en el listado.

6.2.2 Casos de Validación de Errores

Caso de Prueba 14: Error al obtener usuario con ID inexistente

- Endpoint: GET /api/v1/usuarios/{id}
- Método HTTP: GET
- Descripción: Intentar obtener un usuario con un ID que no existe en el sistema.
- Parámetros:
  - Path: id = 9999 (integer).
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 404 Not Found
- Cuerpo de la Respuesta Esperada:

{

"error": "Usuario no encontrado"

}

- Prueba exitosa si: La API devuelve un código 404 Not Found con un mensaje de error indicando que el usuario no existe.

Caso de Prueba 15: Error al actualizar usuario con datos inválidos

- Endpoint: PUT /api/v1/usuarios/{id}
- Método HTTP: PUT
- Descripción: Intentar actualizar un usuario enviando un correo electrónico con formato inválido.
- Parámetros:
  - Path: id = 1 (integer).
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"nombre": "Carlos Pérez",

"email": "correo-invalido"

}

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

{

"error": "El formato del correo electrónico no es válido"

}

- Prueba exitosa si: La API devuelve un código 400 Bad Request indicando que el formato del correo no es válido.

Caso de Prueba 16: Error al eliminar usuario con ID inexistente

- Endpoint: DELETE /api/v1/usuarios/{id}
- Método HTTP: DELETE
- Descripción: Intentar eliminar un usuario con un ID que no existe en el sistema.
- Parámetros:
  - Path: id = 9999 (integer).
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 404 Not Found
- Cuerpo de la Respuesta Esperada:

{

"error": "Usuario no encontrado"

}

- Prueba exitosa si: La API devuelve un código 404 Not Found indicando que el usuario no existe.

6.2.3 Casos de Autenticación y Seguridad

Caso de Prueba 17: Acceder al listado de usuarios sin token

- Endpoint: GET /api/v1/usuarios
- Método HTTP: GET
- Descripción: Verificar que no se puede acceder al listado de usuarios sin autenticación.
- Headers:
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{

"error": "Token de autorización no proporcionado"

}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que se requiere autenticación.

Caso de Prueba 18: Acceder al listado de usuarios con token inválido

- Endpoint: GET /api/v1/usuarios
- Método HTTP: GET
- Descripción: Verificar que no se permite el acceso con un token inválido o expirado.
- Headers:
  - Authorization: Bearer <token_invalido>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{

"error": "Token inválido o expirado"

}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que el token no es válido.

6.3 Módulo de Gastos

Este módulo cubre las operaciones de consulta, creación, actualización y eliminación de gastos registrados por los usuarios. Los casos se dividen en tres grupos: flujos exitosos, validación de errores y seguridad.

6.3.1 Casos de Flujo Exitoso

Caso de Prueba 19: Listar todos los gastos

- Endpoint: GET /api/v1/gastos
- Método HTTP: GET
- Descripción: Obtener la lista completa de gastos registrados en el sistema.
- Parámetros: Ninguno.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

[

{

"id": 1,

"descripcion": "Mercado semanal",

"monto": 250000,

"categoria": "Alimentación",

"fecha": "2026-10-10"

},

{

"id": 2,

"descripcion": "Pago de servicios",

"monto": 180000,

"categoria": "Servicios",

"fecha": "2026-10-12"

}

]

- Prueba exitosa si: La respuesta devuelve el código 200 OK y una lista con todos los gastos registrados.

Caso de Prueba 20: Obtener gasto por ID existente

- Endpoint: GET /api/v1/gastos/{id}
- Método HTTP: GET
- Descripción: Obtener la información de un gasto específico mediante su ID.
- Parámetros:
    - Path: id (integer) → ID del gasto a consultar.
- Headers:

    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"id": 1,

"descripcion": "Mercado semanal",

"monto": 250000,

"categoria": "Alimentación",

"fecha": "2026-10-10"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK con los datos del gasto correspondiente al ID enviado.

Caso de Prueba 21: Crear un nuevo gasto

- Endpoint: POST /api/v1/gastos
- Método HTTP: POST
- Descripción: Registrar un nuevo gasto en el sistema.
- Parámetros: Ninguno.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"descripcion": "Transporte",

"monto": 50000,

"categoria": "Transporte",

"fecha": "2026-10-15"

}

- Código de Respuesta Esperado: 201 Created
- Cuerpo de la Respuesta Esperada:

{

"id": 3,

"descripcion": "Transporte",

"monto": 50000,

"categoria": "Transporte",

"fecha": "2026-10-15"

}

- Prueba exitosa si: La respuesta devuelve el código 201 Created con los datos del gasto creado coincidiendo con los enviados.

Caso de Prueba 22: Actualizar un gasto

- Endpoint: PUT /api/v1/gastos/{id}
- Método HTTP: PUT
- Descripción: Actualizar la información de un gasto existente.
- Parámetros:
  - Path: id (integer) → ID del gasto a actualizar.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"descripcion": "Transporte actualizado",

"monto": 60000,

"categoria": "Transporte",

"fecha": "2026-10-15"

}

- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"id": 3,

"descripcion": "Transporte actualizado",

"monto": 60000,

"categoria": "Transporte",

"fecha": "2026-10-15"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK con los datos actualizados del gasto.

Caso de Prueba 23: Eliminar un gasto

- Endpoint: DELETE /api/v1/gastos/{id}
- Método HTTP: DELETE
- Descripción: Eliminar un gasto del sistema mediante su ID.
- Parámetros:
  - Path: id (integer) → ID del gasto a eliminar.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"message": "Gasto eliminado exitosamente"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK y el gasto ya no aparece en el listado.

6.3.2 Casos de Validación de Errores

Caso de Prueba 24: Error al obtener gasto con ID inexistente

- Endpoint: GET /api/v1/gastos/{id}
- Método HTTP: GET
- Descripción: Intentar obtener un gasto con un ID que no existe en el sistema.
- Parámetros:
  - Path: id = 9999 (integer).
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 404 Not Found
- Cuerpo de la Respuesta Esperada:

{

"error": "Gasto no encontrado"

}

- Prueba exitosa si: La API devuelve un código 404 Not Found indicando que el gasto no existe.

Caso de Prueba 25: Error al crear un gasto con monto inválido

- Endpoint: POST /api/v1/gastos
- Método HTTP: POST
- Descripción: Intentar crear un gasto con un monto negativo o inválido.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"descripcion": "Gasto inválido",

"monto": -5000,

"categoria": "Otros",

"fecha": "2026-10-15"

}

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

{

"error": "El monto debe ser mayor a cero"

}

- Prueba exitosa si: La API devuelve un código 400 Bad Request indicando que el monto no es válido.

Caso de Prueba 26: Error al crear un gasto con campos faltantes

- Endpoint: POST /api/v1/gastos
- Método HTTP: POST
- Descripción: Intentar crear un gasto sin enviar el campo monto.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"descripcion": "Gasto sin monto",

"categoria": "Otros",

"fecha": "2026-10-15"

}

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

{

"error": "El campo monto es obligatorio"

}

- Prueba exitosa si: La API devuelve un código 400 Bad Request indicando que el campo monto es obligatorio.

6.3.3 Casos de Autenticación y Seguridad

Caso de Prueba 27: Acceder al listado de gastos sin token

- Endpoint: GET /api/v1/gastos
- Método HTTP: GET
- Descripción: Verificar que no se puede acceder al listado de gastos sin autenticación.
- Headers:
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{

"error": "Token de autorización no proporcionado"

}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que se requiere autenticación.

Caso de Prueba 28: Acceder al listado de gastos con token inválido

- Endpoint: GET /api/v1/gastos
- Método HTTP: GET
- Descripción: Verificar que no se permite el acceso con un token inválido o expirado.
- Headers:
  - Authorization: Bearer <token_invalido>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{

"error": "Token inválido o expirado"

}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que el token no es válido.

6.4 Módulo de Ingresos

Este módulo cubre las operaciones de consulta, creación, actualización y eliminación de ingresos registrados por los usuarios. Los casos se dividen en tres grupos: flujos exitosos, validación de errores y seguridad.

6.4.1 Casos de Flujo Exitoso

Caso de Prueba 29: Listar todos los ingresos

- Endpoint: GET /api/v1/ingresos
- Método HTTP: GET
- Descripción: Obtener la lista completa de ingresos registrados en el sistema.
- Parámetros: Ninguno.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

[

{

"id": 1,

"descripcion": "Salario mensual",

"monto": 2500000,

"categoria": "Salario",

"fecha": "2026-10-01"

},

{

"id": 2,

"descripcion": "Venta de producto",

"monto": 300000,

"categoria": "Ventas",

"fecha": "2026-10-08"

}

]

- Prueba exitosa si: La respuesta devuelve el código 200 OK y una lista con todos los ingresos registrados.

Caso de Prueba 30: Obtener ingreso por ID existente

- Endpoint: GET /api/v1/ingresos/{id}
- Método HTTP: GET
- Descripción: Obtener la información de un ingreso específico mediante su ID.
- Parámetros:
  - Path: id (integer) → ID del ingreso a consultar.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"id": 1,

"descripcion": "Salario mensual",

"monto": 2500000,

"categoria": "Salario",

"fecha": "2026-10-01"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK con los datos del ingreso correspondiente al ID enviado.

Caso de Prueba 31: Crear un nuevo ingreso

- Endpoint: POST /api/v1/ingresos
- Método HTTP: POST
- Descripción: Registrar un nuevo ingreso en el sistema.
- Parámetros: Ninguno.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"descripcion": "Freelance",

"monto": 800000,

"categoria": "Trabajo adicional",

"fecha": "2026-10-15"

}

- Código de Respuesta Esperado: 201 Created
- Cuerpo de la Respuesta Esperada:

{

"id": 3,

"descripcion": "Freelance",

"monto": 800000,

"categoria": "Trabajo adicional",

"fecha": "2026-10-15"

}

- Prueba exitosa si: La respuesta devuelve el código 201 Created con los datos del ingreso creado coincidiendo con los enviados.

Caso de Prueba 32: Actualizar un ingreso

- Endpoint: PUT /api/v1/ingresos/{id}
- Método HTTP: PUT
- Descripción: Actualizar la información de un ingreso existente.
- Parámetros:
  - Path: id (integer) → ID del ingreso a actualizar.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"descripcion": "Freelance actualizado",

"monto": 900000,

"categoria": "Trabajo adicional",

"fecha": "2026-10-15"

}

- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"id": 3,

"descripcion": "Freelance actualizado",

"monto": 900000,

"categoria": "Trabajo adicional",

"fecha": "2026-10-15"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK con los datos actualizados del ingreso.

Caso de Prueba 33: Eliminar un ingreso

- Endpoint: DELETE /api/v1/ingresos/{id}
- Método HTTP: DELETE
- Descripción: Eliminar un ingreso del sistema mediante su ID.
- Parámetros:
  - Path: id (integer) → ID del ingreso a eliminar.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"message": "Ingreso eliminado exitosamente"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK y el ingreso ya no aparece en el listado.

6.4.2 Casos de Validación de Errores

Caso de Prueba 34: Error al obtener ingreso con ID inexistente

- Endpoint: GET /api/v1/ingresos/{id}
- Método HTTP: GET
- Descripción: Intentar obtener un ingreso con un ID que no existe en el sistema.
- Parámetros:
  - Path: id = 9999 (integer).
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 404 Not Found
- Cuerpo de la Respuesta Esperada:

{

"error": "Ingreso no encontrado"

}

- Prueba exitosa si: La API devuelve un código 404 Not Found indicando que el ingreso no existe.

Caso de Prueba 35: Error al crear un ingreso con monto inválido

- Endpoint: POST /api/v1/ingresos
- Método HTTP: POST

- Descripción: Intentar crear un ingreso con un monto negativo o inválido.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"descripcion": "Ingreso inválido",

"monto": -1000,

"categoria": "Otros",

"fecha": "2026-10-15"

}

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

{

"error": "El monto debe ser mayor a cero"

}

- Prueba exitosa si: La API devuelve un código 400 Bad Request indicando que el monto no es válido.

Caso de Prueba 36: Error al crear un ingreso con campos faltantes

- Endpoint: POST /api/v1/ingresos
- Método HTTP: POST
- Descripción: Intentar crear un ingreso sin enviar el campo monto.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"descripcion": "Ingreso sin monto",

"categoria": "Otros",

"fecha": "2026-10-15"

}

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

{

"error": "El campo monto es obligatorio"

}

- Prueba exitosa si: La API devuelve un código 400 Bad Request indicando que el campo monto es obligatorio.

6.4.3 Casos de Autenticación y Seguridad

Caso de Prueba 37: Acceder al listado de ingresos sin token

- Endpoint: GET /api/v1/ingresos
- Método HTTP: GET
- Descripción: Verificar que no se puede acceder al listado de ingresos sin autenticación.
- Headers:
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{

"error": "Token de autorización no proporcionado"

}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que se requiere autenticación.

Caso de Prueba 38: Acceder al listado de ingresos con token inválido

- Endpoint: GET /api/v1/ingresos
- Método HTTP: GET
- Descripción: Verificar que no se permite el acceso con un token inválido o expirado.
- Headers:
    - Authorization: Bearer <token_invalido>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{

"error": "Token inválido o expirado"

}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que el token no es válido.

6.5 Módulo de Bancos

Este módulo cubre las operaciones de consulta, creación, actualización y eliminación de bancos registrados en el sistema. Los casos se dividen en tres grupos: flujos exitosos, validación de errores y seguridad.

6.5.1 Casos de Flujo Exitoso

Caso de Prueba 39: Listar todos los bancos

- Endpoint: GET /api/v1/bancos
- Método HTTP: GET
- Descripción: Obtener la lista completa de bancos registrados en el sistema.
- Parámetros: Ninguno.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

[

{

"id": 1,

"nombre": "Bancolombia",

"tipo": "Ahorros"

},

{

"id": 2,

"nombre": "Davivienda",

"tipo": "Corriente"

}

]

- Prueba exitosa si: La respuesta devuelve el código 200 OK y una lista con todos los bancos registrados.

Caso de Prueba 40: Obtener banco por ID existente

- Endpoint: GET /api/v1/bancos/{id}
- Método HTTP: GET
- Descripción: Obtener la información de un banco específico mediante su ID.
- Parámetros:
    - Path: id (integer) → ID del banco a consultar.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{
"id": 1,
"nombre": "Bancolombia",
"tipo": "Ahorros"
}

- Prueba exitosa si: La respuesta devuelve el código 200 OK con los datos del banco correspondiente al ID enviado.

Caso de Prueba 41: Crear un nuevo banco

- Endpoint: POST /api/v1/bancos
- Método HTTP: POST
- Descripción: Registrar un nuevo banco en el sistema.
- Parámetros: Ninguno.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"nombre": "Banco de Bogotá",

"tipo": "Ahorros"

}

- Código de Respuesta Esperado: 201 Created
- Cuerpo de la Respuesta Esperada:

{

"id": 3,

"nombre": "Banco de Bogotá",

"tipo": "Ahorros"

}

- Prueba exitosa si: La respuesta devuelve el código 201 Created con los datos del banco creado coincidiendo con los enviados.

Caso de Prueba 42: Actualizar un banco

- Endpoint: PUT /api/v1/bancos/{id}
- Método HTTP: PUT
- Descripción: Actualizar la información de un banco existente.
- Parámetros:
    - Path: id (integer) → ID del banco a actualizar.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"nombre": "Banco de Bogotá Actualizado",

"tipo": "Corriente"

}

- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"id": 3,

"nombre": "Banco de Bogotá Actualizado",

"tipo": "Corriente"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK con los datos actualizados del banco.

Caso de Prueba 43: Eliminar un banco

- Endpoint: DELETE /api/v1/bancos/{id}
- Método HTTP: DELETE
- Descripción: Eliminar un banco del sistema mediante su ID.
- Parámetros:
  - Path: id (integer) → ID del banco a eliminar.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"message": "Banco eliminado exitosamente"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK y el banco ya no aparece en el listado.

6.5.2 Casos de Validación de Errores

Caso de Prueba 44: Error al obtener banco con ID inexistente

- Endpoint: GET /api/v1/bancos/{id}
- Método HTTP: GET
- Descripción: Intentar obtener un banco con un ID que no existe en el sistema.
- Parámetros:
  - Path: id = 9999 (integer).
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 404 Not Found
- Cuerpo de la Respuesta Esperada:

{

"error": "Banco no encontrado"

}

- Prueba exitosa si: La API devuelve un código 404 Not Found indicando que el banco no existe.

Caso de Prueba 45: Error al crear un banco con campos faltantes

- Endpoint: POST /api/v1/bancos
- Método HTTP: POST
- Descripción: Intentar crear un banco sin enviar el campo nombre.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

```
{
"tipo": "Ahorros"
}
```

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

```
{
"error": "El campo nombre es obligatorio"
}
```

- Prueba exitosa si: La API devuelve un código 400 Bad Request indicando que el campo nombre es obligatorio.

Caso de Prueba 46: Error al crear un banco duplicado

- Endpoint: POST /api/v1/bancos
- Método HTTP: POST
- Descripción: Intentar crear un banco con un nombre que ya existe en el sistema.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

```
{
"nombre": "Bancolombia",
"tipo": "Ahorros"
}
```

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

```
{
"error": "El banco ya está registrado"
}
```

- Prueba exitosa si: La API devuelve un código 400 Bad Request indicando que el banco ya existe.

6.5.3 Casos de Autenticación y Seguridad

Caso de Prueba 47: Acceder al listado de bancos sin token

- Endpoint: GET /api/v1/bancos
- Método HTTP: GET
- Descripción: Verificar que no se puede acceder al listado de bancos sin autenticación.
- Headers:
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{

"error": "Token de autorización no proporcionado"

}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que se requiere autenticación.

Caso de Prueba 48: Acceder al listado de bancos con token inválido

- Endpoint: GET /api/v1/bancos
- Método HTTP: GET
- Descripción: Verificar que no se permite el acceso con un token inválido o expirado.
- Headers:
    - Authorization: Bearer <token_invalido>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{

"error": "Token inválido o expirado"

}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que el token no es válido.

6.6 Módulo de Categorías

Este módulo cubre las operaciones de consulta, creación, actualización y eliminación de categorías utilizadas para clasificar gastos e ingresos. Los casos se dividen en tres grupos: flujos exitosos, validación de errores y seguridad.

6.6.1 Casos de Flujo Exitoso

Caso de Prueba 49: Listar todas las categorías

- Endpoint: GET /api/v1/categorias
- Método HTTP: GET
- Descripción: Obtener la lista completa de categorías registradas en el sistema.
- Parámetros: Ninguno.
- Headers:
    - Authorization: Bearer <token>

    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:


```
[
{
"id": 1,
"nombre": "Alimentación",
"tipo": "GASTO"
},
{
"id": 2,
"nombre": "Salario",
"tipo": "INGRESO"
}
]
```


- Prueba exitosa si: La respuesta devuelve el código 200 OK y una lista con todas las categorías registradas.

Caso de Prueba 50: Obtener categoría por ID existente

- Endpoint: GET /api/v1/categorias/{id}
- Método HTTP: GET
- Descripción: Obtener la información de una categoría específica mediante su ID.
- Parámetros:
    - Path: id (integer) → ID de la categoría a consultar.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:


```
{
"id": 1,
"nombre": "Alimentación",
"tipo": "GASTO"
}
```


- Prueba exitosa si: La respuesta devuelve el código 200 OK con los datos de la categoría correspondiente al ID enviado.

Caso de Prueba 51: Crear una nueva categoría

- Endpoint: POST /api/v1/categorias
- Método HTTP: POST
- Descripción: Registrar una nueva categoría en el sistema.

- Parámetros: Ninguno.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"nombre": "Transporte",

"tipo": "GASTO"

}

- Código de Respuesta Esperado: 201 Created
- Cuerpo de la Respuesta Esperada:

{

"id": 3,

"nombre": "Transporte",

"tipo": "GASTO"

}

- Prueba exitosa si: La respuesta devuelve el código 201 Created con los datos de la categoría creada coincidiendo con los enviados.

Caso de Prueba 52: Actualizar una categoría

- Endpoint: PUT /api/v1/categorias/{id}
- Método HTTP: PUT
- Descripción: Actualizar la información de una categoría existente.
- Parámetros:
    - Path: id (integer) → ID de la categoría a actualizar.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"nombre": "Transporte y movilidad",

"tipo": "GASTO"

}

- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"id": 3,

"nombre": "Transporte y movilidad",

"tipo": "GASTO"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK con los datos actualizados de la categoría.

Caso de Prueba 53: Eliminar una categoría

- Endpoint: DELETE /api/v1/categorias/{id}
- Método HTTP: DELETE
- Descripción: Eliminar una categoría del sistema mediante su ID.
- Parámetros:
    - Path: id (integer) → ID de la categoría a eliminar.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 200 OK
- Cuerpo de la Respuesta Esperada:

{

"message": "Categoría eliminada exitosamente"

}

- Prueba exitosa si: La respuesta devuelve el código 200 OK y la categoría ya no aparece en el listado.

6.6.2 Casos de Validación de Errores

Caso de Prueba 54: Error al obtener categoría con ID inexistente

- Endpoint: GET /api/v1/categorias/{id}
- Método HTTP: GET
- Descripción: Intentar obtener una categoría con un ID que no existe en el sistema.
- Parámetros:
    - Path: id = 9999 (integer).
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 404 Not Found
- Cuerpo de la Respuesta Esperada:

{

"error": "Categoría no encontrada"

}

- Prueba exitosa si: La API devuelve un código 404 Not Found indicando que la categoría no existe.

Caso de Prueba 55: Error al crear una categoría con campos faltantes

- Endpoint: POST /api/v1/categorias
- Método HTTP: POST
- Descripción: Intentar crear una categoría sin enviar el campo nombre.
- Headers:
    - Authorization: Bearer <token>
    - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"tipo": "GASTO"

}

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

{

"error": "El campo nombre es obligatorio"

}

- Prueba exitosa si: La API devuelve un código 400 Bad Request indicando que el campo nombre es obligatorio.

Caso de Prueba 56: Error al crear una categoría duplicada

- Endpoint: POST /api/v1/categorias
- Método HTTP: POST
- Descripción: Intentar crear una categoría con un nombre que ya existe en el sistema.
- Headers:
  - Authorization: Bearer <token>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body):

{

"nombre": "Alimentación",

"tipo": "GASTO"

}

- Código de Respuesta Esperado: 400 Bad Request
- Cuerpo de la Respuesta Esperada:

{

"error": "La categoría ya está registrada"

}

- Prueba exitosa si: La API devuelve un código 400 Bad Request indicando que la categoría ya existe.

6.6.3 Casos de Autenticación y Seguridad

Caso de Prueba 57: Acceder al listado de categorías sin token

- Endpoint: GET /api/v1/categorias
- Método HTTP: GET
- Descripción: Verificar que no se puede acceder al listado de categorías sin autenticación.
- Headers:
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{

"error": "Token de autorización no proporcionado"

}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que se requiere autenticación.

Caso de Prueba 58: Acceder al listado de categorías con token inválido

- Endpoint: GET /api/v1/categorias
- Método HTTP: GET
- Descripción: Verificar que no se permite el acceso con un token inválido o expirado.
- Headers:
  - Authorization: Bearer <token_invalido>
  - Content-Type: application/json
- Cuerpo de la Solicitud (Body): No aplica.
- Código de Respuesta Esperado: 401 Unauthorized
- Cuerpo de la Respuesta Esperada:

{

"error": "Token inválido o expirado"

}

- Prueba exitosa si: La API devuelve 401 Unauthorized indicando que el token no es válido.

7. Reporte de Resultados

Una vez ejecutados los casos de prueba, los resultados se documentan en las siguientes tablas, comparando el resultado esperado con el obtenido durante la ejecución. Las tablas están organizadas por módulo para facilitar su seguimiento.

7.1 Módulo de Autenticación

#	Caso de Prueba	Resultado Esperado	Resultado Obtenido	¿Superada?	Observaciones
1	Registro exitoso	201 Created con datos del usuario	Pendiente		
2	Login exitoso	200 OK con token JWT	Pendiente		
3	Logout exitoso	200 OK con mensaje de confirmación	Pendiente		
4	Registro con correo existente	400 Bad Request con mensaje de	Pendiente		

		error			
5	Registro con correo inválido	400 Bad Request con mensaje de error	Pendiente		
6	Registro con campos faltantes	400 Bad Request con mensaje de error	Pendiente		
7	Login con credenciales incorrectas	401 Unauthorized con mensaje genérico	Pendiente		
8	Logout sin token	401 Unauthorized con mensaje de error	Pendiente		
9	Reuso de token finalizado	401 Unauthorized con mensaje de error	Pendiente		

7.2 Módulo de Usuarios

#	Caso de Prueba	Resultado Esperado	Resultado Obtenido	¿Superada?	Observaciones
10	Listar Usuarios	200 OK con lista de usuarios	Pendiente		
11	Obtener usuario por ID	200 OK con datos del usuario	Pendiente		
12	Actualizar usuario	200 OK con datos actualizados	Pendiente		
13	Eliminar usuario	200 OK con mensaje de confirmación	Pendiente		
14	Usuario con ID inexistente	404 Not Found con mensaje de error	Pendiente		
15	Actualizar con datos inválidos	400 Bad Request con mensaje de error	Pendiente		
16	Eliminar usuario	404 Not Found con	Pendiente		

	inexistente	mensaje de error			
17	Listar usuarios sin token	401 Unauthorized con mensaje de error	Pendiente		
18	Listar usuarios con token inválido	401 Unauthorized con mensaje de error	Pendiente		

7.3 Módulo de Gastos

#	Caso de Prueba	Resultado Esperado	Resultado Obtenido	¿Superada?	Observaciones
19	Listar gastos	200 OK con lista de gastos	Pendiente		
20	Obtener gasto por ID	200 OK con datos del gasto	Pendiente		
21	Crear gasto	201 Created con datos del gasto	Pendiente		
22	Actualizar gasto	200 OK con datos actualizados	Pendiente		
23	Eliminar gasto	200 OK con mensaje de confirmación	Pendiente		
24	Gasto con ID inexistente	404 Not Found con mensaje de error	Pendiente		
25	Crear gasto con monto inválido	400 Bad Request con mensaje de error	Pendiente		
26	Crear gasto con campos faltantes	400 Bad Request con mensaje de error	Pendiente		

27	Listar gastos sin token	401 Unauthorized con mensaje de error	Pendiente		
28	Listar gastos con token inválido	401 Unauthorized con mensaje de error	Pendiente		

7.4 Módulo de Ingresos

#	Caso de Prueba	Resultado Esperado	Resultado Obtenido	¿Superada?	Observaciones
29	Listar ingresos	200 OK con lista de ingresos	Pendiente		
30	Obtener ingreso por ID	200 OK con datos del ingreso	Pendiente		
31	Crear ingreso	201 Created con datos del ingreso	Pendiente		
32	Actualizar ingreso	200 OK con datos actualizados	Pendiente		
33	Eliminar ingreso	200 OK con mensaje de confirmación	Pendiente		
34	Ingreso con ID inexistente	404 Not Found con mensaje de error	Pendiente		
35	Crear ingreso con monto inválido	400 Bad Request con mensaje de error	Pendiente		
36	Crear ingreso con campos faltantes	400 Bad Request con mensaje de error	Pendiente		

37	Listar ingresos sin token	401 Unauthorized con mensaje de error	Pendiente		
38	Listar ingresos con token inválido	401 Unauthorized con mensaje de error	Pendiente		

7.5 Módulo de Bancos

#	Caso de Prueba	Resultado Esperado	Resultado Obtenido	¿Superada?	Observaciones
39	Listar bancos	200 OK con lista de bancos	Pendiente		
40	Obtener banco por ID	200 OK con datos del banco	Pendiente		
41	Crear banco	201 Created con datos del banco	Pendiente		
42	Actualizar banco	200 OK con datos actualizados	Pendiente		
43	Eliminar banco	200 OK con mensaje de confirmación	Pendiente		
44	Banco con ID inexistente	404 Not Found con mensaje de error	Pendiente		
45	Crear banco con campos faltantes	400 Bad Request con mensaje de error	Pendiente		
46	Crear banco duplicado	400 Bad Request con mensaje de error	Pendiente		

47	Listar bancos sin token	401 Unauthorized con mensaje de error	Pendiente		
48	Listar bancos con token inválido	401 Unauthorized con mensaje de error	Pendiente		

7.6 Módulo de Categorías

#	Caso de Prueba	Resultado Esperado	Resultado Obtenido	¿Superada?	Observaciones
49	Listar categorías	200 OK con lista de categorías	Pendiente		
50	Obtener categoría por ID	200 OK con datos de la categoría	Pendiente		
51	Crear categoría	201 Created con datos de la categoría	Pendiente		
52	Actualizar categoría	200 OK con datos actualizados	Pendiente		
53	Eliminar categoría	200 OK con mensaje de confirmación	Pendiente		
54	Categoría con ID inexistente	404 Not Found con mensaje de error	Pendiente		
55	Crear categoría con campos faltantes	400 Bad Request con mensaje de error	Pendiente		
56	Crear categoría duplicada	400 Bad Request con mensaje de	Pendiente		

		error			
57	Listar categorías sin token	401 Unauthorized con mensaje de error	Pendiente		
58	Listar categorías con token inválido	401 Unauthorized con mensaje de error	Pendiente		

8. Conclusión

El presente Plan de Pruebas Funcionales reúne un total de 58 casos de prueba distribuidos en los seis módulos que conforman el Sistema de Gestión Financiera Personal: Autenticación, Usuarios, Gastos, Ingresos, Bancos y Categorías.

Cada módulo cuenta con casos que cubren tres frentes: los flujos exitosos que validan el comportamiento esperado de la API, las validaciones de error que aseguran respuestas adecuadas ante solicitudes inválidas, y los controles de seguridad que verifican la protección de los endpoints mediante autenticación con token JWT.

Adicionalmente, el plan define los criterios de aceptación aplicables a todos los módulos, la estrategia de pruebas con los tipos de validación a realizar, las herramientas de ejecución, y las tablas de reporte de resultados donde se registrará el desempeño de cada caso.